PortfolioGenie

Requirements Specification

Functional and Non-Functional Requirements

Digital Egypt Pioneers Initiative (DEPI) - Graduation Project
Phase: Requirements Gathering
Version 1.0 - October 2026

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements of PortfolioGenie. It is the reference for system analysis, design, implementation, and testing, and it is aligned with the User Stories and Use Cases prepared by the team.

1.2 Project Overview

PortfolioGenie is a web platform that helps junior developers build a professional, high-performance portfolio. It analyzes the developer's GitHub activity and uses AI (prompt engineering) to generate tailored content: improved project descriptions (SEO-friendly and readable) and a personal "About Me" section based on the developer's coding style.

1.3 Scope

In scope:

- User registration, authentication and profile management.
- GitHub data import and analysis.
- Multi-step portfolio editor with a live preview.
- AI-assisted content generation and improvement.
- Portfolio publishing and export.

Out of scope: payments, custom domain hosting, and support for platforms other than GitHub (e.g. GitLab, Bitbucket).

1.4 Definitions and Abbreviations

Term	Definition
FR	Functional Requirement
NFR	Non-Functional Requirement
JWT	JSON Web Token, used for stateless authentication
REST API	Web service interface used for communication between frontend and backend
SEO	Search Engine Optimization
WCAG	Web Content Accessibility Guidelines
AI Prompt	Structured instruction sent to the AI model to generate or improve content

2. Overall Description

2.1 Target Users

Actor	Description
Junior Developer (primary)	Registers, connects GitHub, builds, previews, publishes, and exports a portfolio.
Visitor	Views a published portfolio through its public link (no account needed).
GitHub API (external)	Provides repositories, languages and activity data.
AI Service (external)	Generates and improves text content from prompts.

2.2 Technology Constraints (from the program checklist)

- Frontend: React with TypeScript, React Hooks (including Custom Hooks) and React Router.
- Backend: Node.js and Express exposing RESTful APIs.
- Deployment: Docker Compose managing the frontend, backend and database services.
- Documentation: Functional documentation including User Stories and Acceptance Criteria.

2.3 Assumptions and Dependencies

- The user has a public GitHub account with at least one repository.
- The system depends on the availability of the GitHub API and the AI service.
- Internet access is required to use the platform.
- GitHub API rate limits apply and must be handled gracefully.

3. Functional Requirements

Priority levels: High = essential for the first release, Medium = important, Low = nice to have.

3.1 Authentication and Account Management

ID	Requirement	Priority
FR-01	The system shall allow a user to register with name, email and password.	High
FR-02	The system shall allow a registered user to log in and receive a secure JWT session.	High
FR-03	The system shall allow a user to log out, ending the session.	High
FR-04	The system shall allow a user to log in using their GitHub account.	Medium
FR-05	The system shall allow a user to view and update their profile information.	Medium
FR-06	The system shall allow a user to reset a forgotten password.	Low

3.2 GitHub Integration and Analysis

ID	Requirement	Priority
FR-07	The system shall accept a GitHub username (or connected account) and fetch the user's public repositories.	High
FR-08	The system shall analyze repository data: languages used, stars, topics, last activity and commit frequency.	High
FR-09	The system shall display the analysis results as a summary (e.g. top languages and key projects).	Medium
FR-10	The system shall allow the user to select which repositories to include in the portfolio.	High
FR-11	The system shall show a clear message when GitHub data cannot be retrieved (invalid username, rate limit or network error).	High

3.3 Portfolio Editor (Multi-Step Form)

ID	Requirement	Priority
FR-12	The system shall provide a multi-step form covering: Personal Info, Skills, Projects, Experience and Education, and Contact and Social Links.	High
FR-13	The system shall validate the inputs of each step before allowing the user to continue.	High

ID	Requirement	Priority
FR-14	The system shall allow the user to move forward and backward between steps without losing entered data.	High
FR-15	The system shall pre-fill form fields (projects, skills) using the imported GitHub data.	Medium
FR-16	The system shall allow the user to save a draft and resume editing later.	Medium
FR-17	The system shall show a progress indicator for the current step.	Low

3.4 AI Content Generation

ID	Requirement	Priority
FR-18	The system shall improve a project description to be clearer, readable and SEO-friendly using AI.	High
FR-19	The system shall suggest a professional "About Me" section based on the developer's coding style and GitHub activity.	High
FR-20	The system shall let the user accept, edit or reject each AI suggestion.	High
FR-21	The system shall allow the user to regenerate a suggestion.	Medium
FR-22	The system shall show a loading state while the AI response is pending and a clear message if it fails.	High

3.5 Live Preview and Templates

ID	Requirement	Priority
FR-23	The system shall provide a live preview that reflects editor changes without a page reload.	High
FR-24	The system shall use client-side routing to navigate between the editor and the preview.	High
FR-25	The system shall let the user choose from available portfolio templates or themes.	Medium
FR-26	The system shall allow previewing the portfolio in desktop and mobile layouts.	Low

3.6 Publishing and Export

ID	Requirement	Priority
FR-27	The system shall publish the portfolio to a unique public URL.	High
FR-28	The system shall allow the user to unpublish or delete their portfolio.	Medium
FR-29	The system shall allow the user to export the portfolio as static HTML/CSS files.	Medium
FR-30	The system shall allow a visitor to view a published portfolio without logging in.	High

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Pages shall load in under 3 seconds on a standard broadband connection.
NFR-02	Performance	The live preview shall update within 300 ms of an editor change.
NFR-03	Performance	Standard API responses (excluding AI and GitHub calls) shall return within 500 ms for 95% of requests.
NFR-04	Performance	AI-generated content shall be returned within 10 seconds, with a loading indicator shown meanwhile.
NFR-05	Performance	Published portfolios shall be lightweight and achieve a Lighthouse performance score of 90 or above.
NFR-06	Security	Passwords shall be stored hashed (e.g. bcrypt) and never in plain text.
NFR-07	Security	Authentication shall use JWT with an expiry time, and protected routes shall reject unauthenticated requests.
NFR-08	Security	All communication shall use HTTPS in production.
NFR-09	Security	All user inputs shall be validated and sanitized on the server to prevent XSS and injection attacks.
NFR-10	Security	API keys and secrets (AI service, GitHub) shall be kept in environment variables and never exposed to the client.
NFR-11	Security	The API shall apply rate limiting to protect authentication and AI endpoints from abuse.
NFR-12	Usability	The interface shall follow WCAG 2.1 Level AA (contrast, labels, alt text, focus states).
NFR-13	Usability	All main functions shall be usable with a keyboard only.
NFR-14	Usability	The interface shall be responsive on mobile, tablet and desktop screens.
NFR-15	Usability	Error and validation messages shall be clear and tell the user

ID	Category	Requirement
		how to fix the problem.
NFR-16	Usability	A new user shall be able to create a first portfolio draft in under 15 minutes without external help.
NFR-17	Reliability	The system shall target 99% availability during normal operation.
NFR-18	Reliability	The system shall degrade gracefully when GitHub or the AI service is unavailable (manual input remains possible).
NFR-19	Reliability	User drafts shall be saved so that data is not lost on refresh or connection loss.
NFR-20	Reliability	Containers shall restart automatically after a failure.
NFR-21	Maintainability	The code shall be written in TypeScript following clean code practices with ESLint and Prettier.
NFR-22	Maintainability	The code shall be modular (reusable components and Custom Hooks, layered backend).
NFR-23	Maintainability	Core logic shall be covered by automated tests.
NFR-24	Portability	The whole system shall run with a single Docker Compose command (frontend, backend, database).
NFR-25	Compatibility	The platform shall work on the latest versions of Chrome, Edge, Firefox and Safari.

5. Requirements Traceability

This section links requirements to the User Stories and Use Cases prepared by the team, so every requirement is covered and every story maps back to a requirement. The IDs below should be completed once the User Stories and Use Cases are finalized.

5.1 Proposed Use Case Groups

Group	Proposed Use Cases	Related FRs
Account	Register, Log in, Log out, Manage profile	FR-01 to FR-06
GitHub	Import repositories, View analysis, Select repositories	FR-07 to FR-11
Editor	Fill multi-step form, Save draft, Resume draft	FR-12 to FR-17
AI	Improve description, Generate About Me, Accept or reject suggestion	FR-18 to FR-22
Preview	View live preview, Switch template, Preview on mobile	FR-23 to FR-26

Group	Proposed Use Cases	Related FRs
Publish	Publish, Unpublish, Export, View public portfolio	FR-27 to FR-30

5.2 Traceability Matrix

Requirement	User Story ID	Use Case ID	Status
FR-01 to FR-06	TBD	TBD	Pending
FR-07 to FR-11	TBD	TBD	Pending
FR-12 to FR-17	TBD	TBD	Pending
FR-18 to FR-22	TBD	TBD	Pending
FR-23 to FR-26	TBD	TBD	Pending
FR-27 to FR-30	TBD	TBD	Pending